



TIME-AWARE RAG PROJECT - COMPLETE IMPLEMENTATION GUIDE

PROJECT OVERVIEW

What We Built

A **Time-Aware Retrieval Augmented Generation (RAG)** system that answers questions differently based on temporal context. Unlike regular RAG systems that treat all documents as eternally valid, our system understands that information has validity periods.

Real-World Problem

- Policies change over time
- Regulations get updated
- Historical facts need context
- Normal RAG systems return outdated information
- **Our solution:** Filter documents by temporal validity BEFORE generating answers

Example Use Case

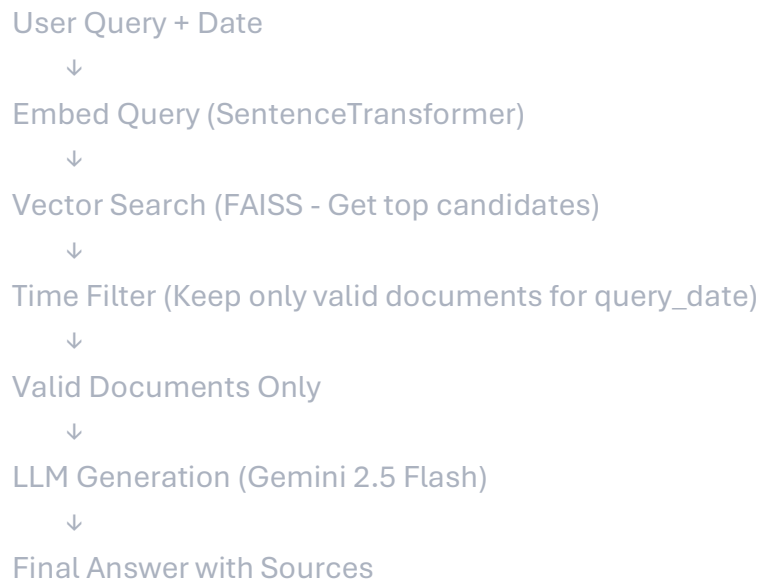
Question: "What is the remote access policy?"

- **Asked on 2022-06-15:** Returns "Use VPN"
- **Asked on 2024-06-15:** Returns "Use zero-trust authentication"

Same question, different dates = different answers!

ARCHITECTURE

System Flow



Key Components

1. **Embeddings:** Convert text to vectors using SentenceTransformer
2. **Vector Store:** Store and search vectors using FAISS
3. **Metadata Store:** Track document validity periods (valid_from, valid_to)
4. **Time-Aware Retriever:** Combine vector search + time filtering
5. **LLM Client:** Generate natural language answers using Gemini
6. **REST API:** FastAPI endpoints for interaction

TECHNOLOGY STACK

Core Technologies

- **Python:** 3.13.7 (or 3.11+ recommended)
- **FastAPI:** Web framework for REST API
- **Uvicorn:** ASGI server
- **Pydantic:** Data validation

ML/AI Components

- **SentenceTransformers:** Text embeddings (all-MiniLM-L6-v2 model)
- **FAISS:** Vector similarity search
- **Gemini 2.5 Flash:** LLM for answer generation
- **PyTorch:** ML backend

Data Storage

- **JSON:** Temporal metadata (Phase 1)
- **FAISS Index:** Vector embeddings
- **PostgreSQL:** Database (Optional Phase 2)

STEP-BY-STEP IMPLEMENTATION

PHASE 0: PROJECT SETUP

Step 1: Create Project Structure

```
bash
mkdir time-aware-rag
cd time-aware-rag
mkdir backend
cd backend
...
```

Create the following directory structure:

```

backend/
├── app/
│   ├── __init__.py
│   ├── main.py           # FastAPI application
│   ├── models.py        # Pydantic schemas
│   ├── config.py        # Settings management
│   └── api/
│       ├── __init__.py
│       └── routes.py     # API endpoints
├── core/
│   ├── __init__.py
│   ├── embeddings.py     # SentenceTransformer wrapper
│   ├── vector_store.py   # FAISS operations
│   ├── metadata_store.py # Temporal metadata manager
│   └── retriever.py      # Time-aware retrieval logic
├── llm/
│   ├── __init__.py
│   └── gemini_client.py  # Gemini API wrapper
├── data/
│   ├── documents/        # Raw documents (optional)
│   ├── metadata.json     # Temporal metadata (generated)
│   └── faiss_index/      # FAISS index files (generated)
├── requirements.txt
├── .env
└── .gitignore

```

Create all directories and empty `__init__.py` files:

bash

```

# Create directories
mkdir -p app/api core llm data/documents data/faiss_index

# Create __init__.py files
touch app/__init__.py app/api/__init__.py core/__init__.py llm/__init__.py

# Create main files
touch app/main.py app/models.py app/config.py app/api/routes.py
touch core/embeddings.py core/vector_store.py core/metadata_store.py core/retriever.py

```

```
touch llm/gemini_client.py
touch requirements.txt .env .gitignore
```

Step 2: Setup Virtual Environment

bash

```
# Create virtual environment
python3 -m venv venv

# Activate it
# On macOS/Linux:
source venv/bin/activate

# On Windows:
# venv\Scripts\activate

# Upgrade pip
pip install --upgrade pip
```

Step 3: Create requirements.txt

txt

```
# Core Framework
fastapi==0.115.6
uvicorn[standard]==0.34.0
pydantic==2.10.5
pydantic-settings==2.7.1

# Embeddings & Vector Search
sentence-transformers==3.3.1
faiss-cpu==1.9.0.post1
numpy==2.2.1
torch>=2.6.0

# LLM
```

```
google-generativeai==0.8.3
```

```
# Database (for Phase 2)
```

```
psycopg2-binary==2.9.10
```

```
sqlalchemy==2.0.36
```

```
# Utilities
```

```
python-dotenv==1.0.1
```

```
python-multipart==0.0.20
```

```
aiofiles==24.1.0
```

```
# Dev Tools
```

```
pytest==8.3.4
```

```
httpx==0.28.1
```

Install dependencies:

```
bash
```

```
pip install -r requirements.txt
```

Note: If you face issues with PyTorch on Python 3.13, downgrade to Python 3.11:

```
bash
```

```
brew install python@3.11
```

```
python3.11 -m venv venv
```

```
source venv/bin/activate
```

```
pip install -r requirements.txt
```

Step 4: Create .env File

Get your Gemini API key from: <https://aistudio.google.com/apikey>

```
env
```

```
# LLM
```

```
GEMINI_API_KEY=your_actual_api_key_here
```

```
# Embedding Model
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2

# Vector Store
FAISS_INDEX_PATH=data/faiss_index/
METADATA_PATH=data/metadata.json

# Server
HOST=0.0.0.0
PORT=8000
```

Important: Replace your_actual_api_key_here with your real API key.

Step 5: Create .gitignore

gitignore

```
# Virtual Environment
venv/
env/
ENV/

# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python

# Environment
.env
.env.local

# Data
data/faiss_index/
data/documents/*.pdf
data/metadata.json
```

```
# IDE
.vscode/
.idea/
*.swp
*.swo

# OS
.DS_Store
Thumbs.db

# Testing
.pytest_cache/
.coverage
htmlcov/
```

PHASE 1: CONFIGURATION & MODELS

Step 6: Create app/config.py

Purpose: Manage environment variables and settings

python

```
from pydantic_settings import BaseSettings
from functools import lru_cache
from pathlib import Path

class Settings(BaseSettings):
    # LLM
    gemini_api_key: str

    # Embeddings
    embedding_model: str = "sentence-transformers/all-MiniLM-L6-v2"

    # Paths
```



```

faiss_index_path: str = "data/faiss_index/"
metadata_path: str = "data/metadata.json"

# Server
host: str = "0.0.0.0"
port: int = 8000

class Config:
    env_file = ".env"
    env_file_encoding = "utf-8"

@lru_cache()
def get_settings():
    return Settings()

# Ensure directories exist
settings = get_settings()
Path(settings.faiss_index_path).mkdir(parents=True, exist_ok=True)
Path(settings.metadata_path).parent.mkdir(parents=True, exist_ok=True)

```

Key Concepts:

- BaseSettings: Automatically reads from .env file
- @lru_cache(): Creates singleton instance (only one Settings object)
- Path().mkdir(): Creates directories if they don't exist

Step 7: Create app/models.py

Purpose: Define request/response schemas using Pydantic

python

```

from pydantic import BaseModel, Field
from typing import Optional, List
from datetime import date

class DocumentUpload(BaseModel):
    content: str = Field(..., description="Document content/text")

```

```

valid_from: str = Field(..., description="Start date (YYYY-MM-DD)")
valid_to: str = Field(..., description="End date (YYYY-MM-DD)")
source: Optional[str] = Field(None, description="Source/filename")
metadata: Optional[dict] = Field(default_factory=dict)

class QueryRequest(BaseModel):
    query: str = Field(..., description="Question to ask")
    query_date: str = Field(..., description="Date context (YYYY-MM-DD)")
    k: int = Field(default=5, description="Number of results to retrieve")

class QueryResponse(BaseModel):
    answer: str
    sources: List[dict]
    query_date: str
    retrieved_count: int

class DocumentResponse(BaseModel):
    doc_id: str
    content: str
    valid_from: str
    valid_to: str
    source: Optional[str]
    created_at: str

class HealthCheck(BaseModel):
    status: str
    embedding_model: str
    faiss_index_exists: bool
    metadata_exists: bool

```

Key Concepts:

- BaseModel: Pydantic base class for data validation
- Field(...): Required field (the ... means "required")
- Field(default=X): Optional field with default value
- These models auto-generate API documentation in FastAPI

PHASE 2: CORE COMPONENTS

Step 8: Create core/embeddings.py

Purpose: Wrapper for SentenceTransformer model

python

```
from sentence_transformers import SentenceTransformer
import numpy as np
from typing import List, Union
from app.config import get_settings

class EmbeddingModel:
    def __init__(self):
        settings = get_settings()
        print(f"📦 Loading embedding model: {settings.embedding_model}")
        self.model = SentenceTransformer(settings.embedding_model)
        self.dimension = self.model.get_sentence_embedding_dimension()
        print(f"✅ Model loaded! Embedding dimension: {self.dimension}")

    def encode(self, texts: Union[str, List[str]], batch_size: int = 32) -> np.ndarray:
        """
        Encode text(s) into embeddings

        Args:
            texts: Single string or list of strings
            batch_size: Batch size for encoding

        Returns:
            numpy array of embeddings
        """
        if isinstance(texts, str):
            texts = [texts]

        embeddings = self.model.encode(
            texts,
```

```

        batch_size=batch_size,
        show_progress_bar=len(texts) > 10,
        convert_to_numpy=True
    )

    return embeddings

def get_dimension(self) -> int:
    """Get embedding dimension"""
    return self.dimension

# Singleton instance
_embedding_model = None

def get_embedding_model() -> EmbeddingModel:
    """Get or create embedding model singleton"""
    global _embedding_model
    if _embedding_model is None:
        _embedding_model = EmbeddingModel()
    return _embedding_model
` ``

**Key Concepts:**
- **Embeddings:** Convert text to numerical vectors (384 dimensions for all-MiniLM-L6-v2)
- **Singleton Pattern:** Only load model once (expensive operation)
- **Batch Processing:** Process multiple texts efficiently
- **Model:** all-MiniLM-L6-v2 (fast, efficient, good for semantic search)

**How Embeddings Work:**
` ``

"What is the policy?" -> [0.23, -0.45, 0.67, ..., 0.12] (384 numbers)
"Policy information" -> [0.21, -0.43, 0.65, ..., 0.15] (384 numbers)
    ↑ Similar vectors = Similar meaning

```

Step 9: Create core/vector_store.py

Purpose: Manage FAISS vector index for similarity search

python

```
import faiss
import numpy as np
import os
import pickle
from typing import List, Tuple, Optional
from app.config import get_settings

class VectorStore:
    def __init__(self):
        self.settings = get_settings()
        self.index = None
        self.dimension = None
        self.doc_ids = [] # Keep track of document IDs

    # Paths
    self.index_path = os.path.join(self.settings.faiss_index_path, "index.faiss")
    self.doc_ids_path = os.path.join(self.settings.faiss_index_path, "doc_ids.pkl")

    print("📁 VectorStore initialized")

    def create_index(self, dimension: int):
        """Create a new FAISS index"""
        self.dimension = dimension
        # Using IndexFlatL2 for exact search (good for small-medium datasets)
        self.index = faiss.IndexFlatL2(dimension)
        print(f"✅ Created FAISS index with dimension: {dimension}")

    def add_vectors(self, embeddings: np.ndarray, doc_ids: List[str]):
        """
        Add vectors to the index

        Args:
            embeddings: numpy array of shape (n, dimension)
            doc_ids: list of document IDs corresponding to embeddings
        """
        if self.index is None:
```

```

self.create_index(embeddings.shape[1])

# Ensure correct shape
if len(embeddings.shape) == 1:
    embeddings = embeddings.reshape(1, -1)

# Add to FAISS
self.index.add(embeddings.astype('float32'))
self.doc_ids.extend(doc_ids)

print(f"✅ Added {len(doc_ids)} vectors to index. Total: {self.index.ntotal}")

def search(self, query_embedding: np.ndarray, k: int = 5) -> Tuple[List[float], List[str]]:
    """
    Search for similar vectors

    Args:
        query_embedding: query vector
        k: number of results to return

    Returns:
        Tuple of (distances, doc_ids)
    """
    if self.index is None or self.index.ntotal == 0:
        print("⚠️ Index is empty!")
        return [], []

    # Ensure correct shape
    if len(query_embedding.shape) == 1:
        query_embedding = query_embedding.reshape(1, -1)

    # Search
    k = min(k, self.index.ntotal) # Don't request more than available
    distances, indices = self.index.search(query_embedding.astype('float32'), k)

    # Get doc_ids for the indices
    result_doc_ids = [self.doc_ids[idx] for idx in indices[0]]

```

```

result_distances = distances[0].tolist()

print(f"🔍 Found {len(result_doc_ids)} results")
return result_distances, result_doc_ids

def save(self):
    """Save index and doc_ids to disk"""
    if self.index is None:
        print("⚠️ No index to save")
        return

    os.makedirs(self.settings.faiss_index_path, exist_ok=True)

    # Save FAISS index
    faiss.write_index(self.index, self.index_path)

    # Save doc_ids
    with open(self.doc_ids_path, 'wb') as f:
        pickle.dump(self.doc_ids, f)

    print(f"💾 Saved index to {self.index_path}")

def load(self) -> bool:
    """Load index and doc_ids from disk"""
    if not os.path.exists(self.index_path):
        print("⚠️ No saved index found")
        return False

    try:
        # Load FAISS index
        self.index = faiss.read_index(self.index_path)
        self.dimension = self.index.d

        # Load doc_ids
        with open(self.doc_ids_path, 'rb') as f:
            self.doc_ids = pickle.load(f)

```

```

        print(f"✅ Loaded index with {self.index.ntotal} vectors")
        return True
    except Exception as e:
        print(f"❌ Error loading index: {e}")
        return False

def get_stats(self) -> dict:
    """Get index statistics"""
    return {
        "total_vectors": self.index.ntotal if self.index else 0,
        "dimension": self.dimension,
        "doc_count": len(self.doc_ids)
    }

# Singleton instance
_vector_store = None

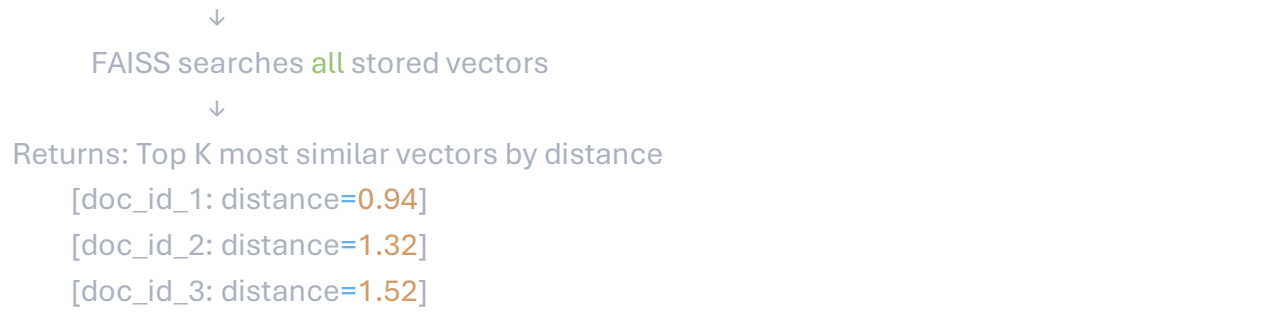
def get_vector_store() -> VectorStore:
    """Get or create vector store singleton"""
    global _vector_store
    if _vector_store is None:
        _vector_store = VectorStore()
        _vector_store.load() # Try to load existing index
    return _vector_store
` ``

**Key Concepts:**
- **FAISS:** Facebook AI Similarity Search - ultra-fast vector search library
- **IndexFlatL2:** Exact search using L2 (Euclidean) distance
- **L2 Distance:** Lower score = more similar (0 = identical)
- **Persistence:** Save/load index to disk for reuse
- **doc_ids:** Map FAISS indices back to document IDs

**How FAISS Works:**
` ``

Query: "login policy" → [0.23, -0.45, 0.67, ...]

```

Step 10: Create core/metadata_store.py

Purpose: Store and manage temporal metadata for documents

python

```
import json
import os
from typing import List, Dict, Optional
from datetime import datetime
from app.config import get_settings
import uuid

class MetadataStore:
    def __init__(self):
        self.settings = get_settings()
        self.metadata_path = self.settings.metadata_path
        self.metadata = {}
        self.load()

        print("📄 MetadataStore initialized")

    def add_document(
        self,
        content: str,
        valid_from: str,
        valid_to: str,
        source: Optional[str] = None,
        metadata: Optional[dict] = None
    ) -> str:
```

```
"""
```

Add a document with temporal metadata

Args:

content: Document text
valid_from: Start date (YYYY-MM-DD)
valid_to: End date (YYYY-MM-DD)
source: Source filename/reference
metadata: Additional metadata

Returns:

doc_id: Generated document ID

```
"""
```

```
doc_id = str(uuid.uuid4())
```

```
self.metadata[doc_id] = {  
    "doc_id": doc_id,  
    "content": content,  
    "valid_from": valid_from,  
    "valid_to": valid_to,  
    "source": source or "unknown",  
    "created_at": datetime.now().isoformat(),  
    "metadata": metadata or {}  
}
```

```
print(f"✅ Added document: {doc_id[:8]}...")  
return doc_id
```

```
def get_document(self, doc_id: str) -> Optional[Dict]:  
    """Get document by ID"""  
    return self.metadata.get(doc_id)
```

```
def get_all_documents(self) -> List[Dict]:  
    """Get all documents"""  
    return list(self.metadata.values())
```

```
def get_valid_documents(self, query_date: str) -> List[Dict]:  
    """
```

Get documents valid on a specific date

Args:

query_date: Date to check (YYYY-MM-DD)

Returns:

List of valid documents

"""

valid_docs = []

for doc in self.metadata.values():

if doc["valid_from"] <= query_date <= doc["valid_to"]:

valid_docs.append(doc)

print(f"✅ Found {len(valid_docs)} valid documents for date: {query_date}")

return valid_docs

def filter_by_ids(self, doc_ids: List[str]) -> List[Dict]:

"""Get documents by list of IDs"""

return [self.metadata[doc_id] for doc_id in doc_ids if doc_id in self.metadata]

def delete_document(self, doc_id: str) -> bool:

"""Delete a document"""

if doc_id in self.metadata:

del self.metadata[doc_id]

print(f"🗑 Deleted document: {doc_id[:8]}...")

return True

return False

def save(self):

"""Save metadata to disk"""

os.makedirs(os.path.dirname(self.metadata_path), exist_ok=True)

with open(self.metadata_path, 'w') as f:

json.dump(self.metadata, f, indent=2)

print(f"💾 Saved metadata: {len(self.metadata)} documents")

```

def load(self):
    """Load metadata from disk"""
    if not os.path.exists(self.metadata_path):
        print(" ⚠ No metadata file found, starting fresh")
        self.metadata = {}
        return

    try:
        with open(self.metadata_path, 'r') as f:
            self.metadata = json.load(f)
        print(f" ✅ Loaded metadata: {len(self.metadata)} documents")
    except Exception as e:
        print(f" ❌ Error loading metadata: {e}")
        self.metadata = {}

def get_stats(self) -> dict:
    """Get metadata statistics"""
    return {
        "total_documents": len(self.metadata),
        "sources": list(set(doc["source"] for doc in self.metadata.values()))
    }

# Singleton instance
_metadata_store = None

def get_metadata_store() -> MetadataStore:
    """Get or create metadata store singleton"""
    global _metadata_store
    if _metadata_store is None:
        _metadata_store = MetadataStore()
    return _metadata_store

```

Key Concepts:

- **Temporal Metadata:** Each document has valid_from and valid_to dates
- **UUID:** Generates unique IDs for each document
- **JSON Storage:** Simple, human-readable storage (Phase 1)

- **Time Filtering:** Core logic for checking if document is valid on a date

Metadata Structure:

json

```
{
  "doc_id_123": {
    "doc_id": "doc_id_123",
    "content": "Users must use VPN...",
    "valid_from": "2022-01-01",
    "valid_to": "2023-12-31",
    "source": "policy.pdf",
    "created_at": "2024-01-15T10:30:00"
  }
}
```

Step 11: Create core/retriever.py

Purpose: THE BRAIN - Orchestrates time-aware retrieval

python

```
from typing import List, Dict, Tuple
from core.embeddings import get_embedding_model
from core.vector_store import get_vector_store
from core.metadata_store import get_metadata_store

class TimeAwareRetriever:
    def __init__(self):
        self.embedder = get_embedding_model()
        self.vector_store = get_vector_store()
        self.metadata_store = get_metadata_store()
        print("🧠 TimeAwareRetriever initialized")

    def add_document(
        self,
        content: str,
        valid_from: str,
```

```
valid_to: str,  
source: str = None,  
metadata: dict = None
```

```
) -> str:
```

```
"""
```

Add a document to the system

Args:

content: Document text
valid_from: Start date (YYYY-MM-DD)
valid_to: End date (YYYY-MM-DD)
source: Source reference
metadata: Additional metadata

Returns:

doc_id: Document ID

```
"""
```

1. Add to metadata store

```
doc_id = self.metadata_store.add_document(  
    content=content,  
    valid_from=valid_from,  
    valid_to=valid_to,  
    source=source,  
    metadata=metadata  
)
```

2. Generate embedding

```
embedding = self.embedder.encode(content)
```

3. Add to vector store

```
self.vector_store.add_vectors(embedding, [doc_id])
```

4. Save both stores

```
self.metadata_store.save()  
self.vector_store.save()
```

```
print(f"✅ Document added successfully: {doc_id[:8]}...")
```

```
return doc_id
```

```
def retrieve(
    self,
    query: str,
    query_date: str,
    k: int = 5
) -> Tuple[List[Dict], List[float]]:
    """
```

Time-aware retrieval

Args:

query: Search query
 query_date: Date context (YYYY-MM-DD)
 k: Number of results

Returns:

Tuple of (documents, scores)

"""

```
print(f"\n🔍 Retrieving for query: '{query}' on date: {query_date}")
```

1. Embed query

```
query_embedding = self.embedder.encode(query)
```

2. Get top candidates from FAISS (get more than k for filtering)

```
search_k = min(k * 3, self.vector_store.index.ntotal) if self.vector_store.index else k
distances, candidate_ids = self.vector_store.search(query_embedding, k=search_k)
```

```
if not candidate_ids:
```

```
    print("⚠️ No candidates found in vector store")
```

```
    return [], []
```

```
print(f"📦 Retrieved {len(candidate_ids)} candidates from vector store")
```

3. Get metadata for candidates

```
candidates = self.metadata_store.filter_by_ids(candidate_ids)
```

4. Filter by time validity

```

valid_docs = []
valid_scores = []

for doc, distance in zip(candidates, distances):
    if doc["valid_from"] <= query_date <= doc["valid_to"]:
        valid_docs.append(doc)
        # Convert L2 distance to similarity score (lower is better)
        valid_scores.append(float(distance))

# 5. Sort by score and limit to k
sorted_pairs = sorted(zip(valid_docs, valid_scores), key=lambda x: x[1])
valid_docs = [doc for doc, _ in sorted_pairs[:k]]
valid_scores = [score for _, score in sorted_pairs[:k]]

print(f"✅ Found {len(valid_docs)} time-valid documents")

return valid_docs, valid_scores

def get_all_documents(self) -> List[Dict]:
    """Get all documents"""
    return self.metadata_store.get_all_documents()

def delete_document(self, doc_id: str) -> bool:
    """Delete a document (metadata only, FAISS cleanup requires rebuild)"""
    success = self.metadata_store.delete_document(doc_id)
    if success:
        self.metadata_store.save()
    return success

# Singleton instance
_retriever = None

def get_retriever() -> TimeAwareRetriever:
    """Get or create retriever singleton"""
    global _retriever
    if _retriever is None:
        _retriever = TimeAwareRetriever()
    return _retriever

```



```\n\n\*\*Key Concepts:\*\*

- \*\*Two-Stage Retrieval:\*\*

1. Vector similarity search (FAISS)
  2. Temporal filtering (date check)
- \*\*Over-fetching:\*\* Get `k \* 3` candidates from FAISS, then filter to `k`
- \*\*Why?\*\* Some candidates may be filtered out by date
- \*\*Orchestration:\*\* Coordinates embeddings, vector store, and metadata

\*\*Retrieval Flow:\*\*

- ```\n\n1. Query: "What is the policy?" + Date: "2023-06-15"
2. Embed query → [0.23, -0.45, 0.67, ...]
3. FAISS search → Get top 15 similar docs
4. Time filter → Keep only docs valid on 2023-06-15
- ☑ Doc A: 2023-01-01 to 2024-12-31 (KEEP)
  - ✗ Doc B: 2021-01-01 to 2022-12-31 (DISCARD)
  - ☑ Doc C: 2023-06-01 to 2024-06-30 (KEEP)
5. Return top 5 time-valid docs

## PHASE 3: LLM INTEGRATION

### Step 12: Create llm/gemini\_client.py

**Purpose:** Wrapper for Gemini API to generate natural language answers

python

```
import google.generativeai as genai
from typing import List, Dict
from app.config import get_settings

class GeminiClient:
 def __init__(self):
 settings = get_settings()
 genai.configure(api_key=settings.gemini_api_key)
```

```

Use Gemini 2.5 Flash (latest and fastest)
self.model = genai.GenerativeModel('gemini-2.5-flash')
print("🧠 Gemini client initialized with gemini-2.5-flash")

def generate_answer(
 self,
 query: str,
 context_docs: List[Dict],
 query_date: str
) -> str:
 """
 Generate answer using retrieved context

 Args:
 query: User's question
 context_docs: Retrieved documents
 query_date: Date context

 Returns:
 Generated answer
 """

 # Build context from documents
 if not context_docs:
 return "I couldn't find any relevant information in the valid documents for the specified date."

 context_text = "\n\n".join([
 f"Document {i+1} (Valid: {doc['valid_from']} to {doc['valid_to']}):\n{doc['content']}"
 for i, doc in enumerate(context_docs)
])

 # Create prompt
 prompt = f"""You are a helpful assistant that answers questions based ONLY on the provided context documents.

 Current Date Context: {query_date}

```

Context Documents:

{context\_text}

User Question: {query}

Instructions:

1. Answer based ONLY on the information in the provided documents
2. If the documents don't contain enough information to answer, say "The provided documents don't contain sufficient information to answer this question."
3. Be concise and direct
4. If multiple documents have conflicting information, mention the validity periods
5. Do not make up information or use knowledge outside the provided context

Answer:"""

```
try:
 response = self.model.generate_content(prompt)
 return response.text
except Exception as e:
 print(f"❌ Error generating answer: {e}")
 return f"Error generating answer: {str(e)}"
```

*# Singleton instance*

\_gemini\_client = None

def get\_gemini\_client() -> GeminiClient:

"""Get or create Gemini client singleton"""

global \_gemini\_client

if \_gemini\_client is None:

\_gemini\_client = GeminiClient()

return \_gemini\_client

...

**\*\*Key Concepts:\*\***

- **\*\*LLM Role:\*\*** Generate natural language answers from retrieved context
- **\*\*Grounding:\*\*** LLM only uses provided context (prevents hallucination)
- **\*\*Prompt Engineering:\*\*** Clear instructions to stay within context

- **Error Handling:** Graceful fallback on API errors

**Why LLM at the End?**

...

Without LLM: Return raw document chunks (user has to read)

With LLM: Synthesize information into natural answer

Example:

Retrieved: ["Doc1: Use VPN", "Doc2: Enable 2FA"]

LLM Output: "To access remotely, use VPN and enable 2FA."

## PHASE 4: API LAYER

### Step 13: Create app/main.py

**Purpose:** Main FastAPI application

python

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.config import get_settings
from app.models import HealthCheck
from app.api import routes
import os

settings = get_settings()

app = FastAPI(
 title="Time-Aware RAG API",
 description="Retrieval Augmented Generation with Temporal Intelligence",
 version="1.0.0"
)

CORS
app.add_middleware(
 CORSMiddleware,
```

```

 allow_origins=["*"],
 allow_credentials=True,
 allow_methods=["*"],
 allow_headers=["*"],
)

Include routes
app.include_router(routes.router)

@app.get("/", response_model=HealthCheck)
async def health_check():
 """Health check endpoint"""
 faiss_index_exists = os.path.exists(os.path.join(settings.faiss_index_path, "index.faiss"))
 metadata_exists = os.path.exists(settings.metadata_path)

 return HealthCheck(
 status="healthy",
 embedding_model=settings.embedding_model,
 faiss_index_exists=faiss_index_exists,
 metadata_exists=metadata_exists
)

if __name__ == "__main__":
 import uvicorn
 uvicorn.run(
 "app.main:app",
 host=settings.host,
 port=settings.port,
 reload=True
)

```

### Key Concepts:

- **FastAPI App:** Main application instance
- **CORS:** Allow cross-origin requests (for frontend)
- **Health Check:** Verify system status
- **Auto-reload:** Development mode (changes restart server)

## Step 14: Create app/api/routes.py

**Purpose:** API endpoints

python

```
from fastapi import APIRouter, HTTPException
from app.models import DocumentUpload, QueryRequest, QueryResponse, DocumentResponse
from typing import List
from core.retriever import get_retriever
from llm.gemini_client import get_gemini_client

router = APIRouter(prefix="/api", tags=["RAG"])

@router.post("/upload", response_model=dict)
async def upload_document(doc: DocumentUpload):
 """Upload a document with temporal metadata"""
 try:
 retriever = get_retriever()

 doc_id = retriever.add_document(
 content=doc.content,
 valid_from=doc.valid_from,
 valid_to=doc.valid_to,
 source=doc.source,
 metadata=doc.metadata
)

 return {
 "status": "success",
 "doc_id": doc_id,
 "message": "Document uploaded successfully"
 }
 except Exception as e:
 raise HTTPException(status_code=500, detail=str(e))

@router.post("/query", response_model=QueryResponse)
async def query_documents(request: QueryRequest):
```

```

"""Query documents with time-aware retrieval and LLM generation"""
try:
 retriever = get_retriever()
 gemini = get_gemini_client()

 # Retrieve relevant documents
 docs, scores = retriever.retrieve(
 query=request.query,
 query_date=request.query_date,
 k=request.k
)

 # Generate answer using LLM
 answer = gemini.generate_answer(
 query=request.query,
 context_docs=docs,
 query_date=request.query_date
)

 # Prepare sources
 sources = [
 {
 "doc_id": doc["doc_id"],
 "content": doc["content"][:200] + "..." if len(doc["content"]) > 200 else
doc["content"],
 "valid_from": doc["valid_from"],
 "valid_to": doc["valid_to"],
 "source": doc["source"],
 "score": float(score)
 }
 for doc, score in zip(docs, scores)
]

 return QueryResponse(
 answer=answer,
 sources=sources,
 query_date=request.query_date,
 retrieved_count=len(docs)
)

```

```

)
except Exception as e:
 raise HTTPException(status_code=500, detail=str(e))

@router.get("/documents", response_model=List[DocumentResponse])
async def list_documents():
 """List all documents with metadata"""
 try:
 retriever = get_retriever()
 docs = retriever.get_all_documents()

 return [
 DocumentResponse(
 doc_id=doc["doc_id"],
 content=doc["content"],
 valid_from=doc["valid_from"],
 valid_to=doc["valid_to"],
 source=doc.get("source", "unknown"),
 created_at=doc["created_at"]
)
 for doc in docs
]
 except Exception as e:
 raise HTTPException(status_code=500, detail=str(e))

@router.delete("/documents/{doc_id}")
async def delete_document(doc_id: str):
 """Delete a document"""
 try:
 retriever = get_retriever()
 success = retriever.delete_document(doc_id)

 if not success:
 raise HTTPException(status_code=404, detail="Document not found")

 return {
 "status": "success",
 "message": f"Document {doc_id} deleted successfully"
 }
 except Exception as e:
 raise HTTPException(status_code=500, detail=str(e))

```



```

 }
except HTTPException:
 raise
except Exception as e:
 raise HTTPException(status_code=500, detail=str(e))

@router.get("/stats")
async def get_stats():
 """Get system statistics"""
 retriever = get_retriever()
 vector_stats = retriever.vector_store.get_stats()
 metadata_stats = retriever.metadata_store.get_stats()

 return {
 "vector_store": vector_stats,
 "metadata_store": metadata_stats
 }

```

### Key Endpoints:

1. **POST /api/upload:** Add new document
2. **POST /api/query:** Ask questions with date context
3. **GET /api/documents:** List all documents
4. **DELETE /api/documents/{id}:** Remove document
5. **GET /api/stats:** System statistics

## PHASE 5: RUNNING & TESTING

### Step 15: Run the Server

bash

```

Make sure you're in the backend/ directory with venv activated
python -m app.main

```

**OR:**

bash

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

...

**\*\*Expected Output:\*\***

...

 Loading embedding model: sentence-transformers/all-MiniLM-L6-v2

 Model loaded! Embedding dimension: 384

 VectorStore initialized

 No saved index found

 MetadataStore initialized

 No metadata file found, starting fresh

 TimeAwareRetriever initialized

INFO: Uvicorn running on <http://0.0.0.0:8000> (Press CTRL+C to quit)

INFO: Started reloader process

INFO: Started server process

INFO: Waiting for application startup.

INFO: Application startup complete.

## Step 16: Test Health Check

**Open browser:** <http://localhost:8000>

**Expected Response:**

```
json
{
 "status": "healthy",
 "embedding_model": "sentence-transformers/all-MiniLM-L6-v2",
 "faiss_index_exists": false,
 "metadata_exists": false
}
```

## Step 17: Interactive API Documentation

**Open:** <http://localhost:8000/docs>

You'll see Swagger UI with all endpoints!

# PHASE 6: TESTING THE SYSTEM

## Test 1: Upload Documents

**Endpoint:** POST /api/upload

### Document 1 (2022 Policy):

json

```
{
 "content": "All employees must use VPN when accessing company resources remotely. This policy is effective from January 2022.",
 "valid_from": "2022-01-01",
 "valid_to": "2023-12-31",
 "source": "security_policy_2022.pdf"
}
```

### Document 2 (2024 Policy):

json

```
{
 "content": "Updated security policy: VPN is no longer required. Instead, use our new zero-trust authentication system with hardware tokens. Effective from January 2024.",
 "valid_from": "2024-01-01",
 "valid_to": "2025-12-31",
 "source": "security_policy_2024.pdf"
}
```

### Document 3 (Login Requirements):

json

```
{
 "content": "Login requires username, password, and 2FA code. All users must enable 2FA by March 2023.",
 "valid_from": "2023-01-01",
}
```

```
"valid_to": "2024-12-31",
"source": "login_requirements.pdf"
}
```

### Expected Response (for each):

json

```
{
 "status": "success",
 "doc_id": "91c812e5-06db-4c2b-a44e-d3e98d1b273e",
 "message": "Document uploaded successfully"
}
```

## Test 2: Query with 2022 Date

**Endpoint:** POST /api/query

### Request:

json

```
{
 "query": "What is the remote access policy?",
 "query_date": "2022-06-15",
 "k": 5
}
```

### Expected Response:

json

```
{
 "answer": "All employees must use VPN when accessing company resources remotely.",
 "sources": [
 {
 "doc_id": "7ae197be-63a5-4417-bf83-d528b0e4ff7c",
 "content": "All employees must use VPN when accessing company resources remotely. This policy is effective from January 2022.",
 }
]
}
```

```
{
 "valid_from": "2022-01-01",
 "valid_to": "2023-12-31",
 "source": "security_policy_2022.pdf",
 "score": 0.9436280727386475
},
{
 "query_date": "2022-06-15",
 "retrieved_count": 1
}
```

### Key Observation:

- ☒ Retrieved 2022 VPN policy
- ☒ Answer mentions VPN
- ☒ Did NOT retrieve 2024 policy (filtered by date)

### Test 3: Query with 2024 Date

**Endpoint:** POST /api/query

#### Request:

json

```
{
 "query": "What is the remote access policy?",
 "query_date": "2024-06-15",
 "k": 5
}
```

#### Expected Response:

json

```
{
 "answer": "For remote access, VPN is no longer required. Instead, use the new zero-trust authentication system with hardware tokens, effective from January 2024.",
 "sources": [
```

```
{
 "doc_id": "e7f59a55-a1e8-43c5-a899-226467571f25",
 "content": "Updated security policy: VPN is no longer required. Instead, use our new zero-trust authentication system with hardware tokens...",
 "valid_from": "2024-01-01",
 "valid_to": "2025-12-31",
 "source": "security_policy_2024.pdf",
 "score": 1.3206183910369873
}
],
"query_date": "2024-06-15",
"retrieved_count": 1
}
```

### Key Observation:

- ☒ Retrieved 2024 zero-trust policy
- ☒ Answer mentions zero-trust authentication
- ☒ Did NOT retrieve 2022 VPN policy (filtered by date)

**THIS IS THE MAGIC!** 🌈 Same question, different dates, different answers!

## Test 4: List All Documents

**Endpoint:** GET /api/documents

### Expected Response:

json

```
[
 {
 "doc_id": "7ae197be-63a5-4417-bf83-d528b0e4ff7c",
 "content": "All employees must use VPN...",
 "valid_from": "2022-01-01",
 "valid_to": "2023-12-31",
 "source": "security_policy_2022.pdf",
 }
]
```

```
"created_at": "2024-01-15T10:30:00"
},
{
 "doc_id": "e7f59a55-a1e8-43c5-a899-226467571f25",
 "content": "Updated security policy: VPN is no longer required...",
 "valid_from": "2024-01-01",
 "valid_to": "2025-12-31",
 "source": "security_policy_2024.pdf",
 "created_at": "2024-01-15T10:31:00"
}
]
```

## Test 5: System Statistics

**Endpoint:** GET /api/stats

**Expected Response:**

json

```
{
 "vector_store": {
 "total_vectors": 3,
 "dimension": 384,
 "doc_count": 3
 },
 "metadata_store": {
 "total_documents": 3,
 "sources": [
 "security_policy_2022.pdf",
 "security_policy_2024.pdf",
 "login_requirements.pdf"
]
 }
}
```



# UNDERSTANDING THE SYSTEM



## How Time-Aware RAG Works

### ◇ Example Scenario

- **User Query:** “What is the remote access policy?”
- **Query Date:** 2024-06-15

## 1 Embedding Generation

- User query is converted into a numerical vector using a **SentenceTransformer**.
- This vector represents the semantic meaning of the query.

Query → SentenceTransformer → [0.23, -0.45, 0.67, ..., 0.12]

## 2 Vector Search (FAISS)

- The query embedding is searched against a **FAISS vector index**.
- FAISS returns the **top 15 most similar documents** based on vector distance.

Top Results:

- Doc1 (2022 VPN) → distance = 0.94
- Doc2 (2024 Zero-trust) → distance = 1.32
- Doc3 (Login) → distance = 1.52

## 3 Temporal Filtering

- Each retrieved document is checked against the **query date (2024-06-15)**.
- Only documents **valid during the query date** are kept.



### Doc1 (2022 VPN)

- valid\_from: 2022-01-01
- valid\_to: 2023-12-31
- 2024-06-15  Not in range → DISCARD

### Doc2 (2024 Zero-trust)

- valid\_from: 2024-01-01
- valid\_to: 2025-12-31
- 2024-06-15  In range → KEEP

### Doc3 (Login)

- valid\_from: 2023-01-01
- valid\_to: 2024-12-31
- 2024-06-15  In range → KEEP

## LLM Generation (Gemini)

- Only **time-valid documents** are sent to the LLM.
- Strict instruction: **Answer using ONLY the provided context.**

Sent to Gemini:

- Context: Doc2 + Doc3
- Query: "What is the remote access policy?"
- Instruction: Use only the given context

### Gemini Output

"For remote access, VPN is no longer required. Instead, use the new zero-trust authentication system with hardware tokens."

## 5 Final Response Returned

- The system returns the answer along with metadata.

```
{
 "answer": "...",
 "sources": ["Doc2", "Doc3"],
 "query_date": "2024-06-15",
 "retrieved_count": 2
}
```

## Why This Approach Works

### Problem with Normal RAG:

- Retrieves all similar documents regardless of time
- LLM gets conflicting information (2022 policy + 2024 policy)
- May hallucinate or give wrong answer

### Solution with Time-Aware RAG:

- Filter documents by temporal validity BEFORE LLM
- LLM only sees documents valid at query date
- Guarantees time-correct answers

## Key Design Decisions

### 1. Why FAISS + JSON instead of Vector Database?

- **Learning:** Understand core concepts without abstractions
- **Control:** Full control over indexing and retrieval
- **Simplicity:** Easy to debug and understand
- **Performance:** FAISS is extremely fast for small-medium datasets

### 2. Why Separate Vector Store and Metadata Store?

- **Modularity:** Each component has single responsibility
- **Flexibility:** Easy to swap JSON → PostgreSQL later
- **Efficiency:** FAISS only stores vectors, metadata stored separately

### 3. Why Retrieve More Candidates ( $k * 3$ )?

- Some candidates will be filtered by date
- Ensures we still return  $k$  results after filtering
- Trade-off: More computation vs. better recall

### 4. Why Singleton Pattern?

- Embedding model is expensive to load (takes ~2-3 seconds)
- Load once, reuse across all requests
- Saves memory and time

### 5. Why LLM at the End?

- Retrieval finds relevant docs (fast, deterministic)
- LLM synthesizes information (slower, creative)
- Separation of concerns: retrieval vs. generation

## INTERVIEW TALKING POINTS

### Technical Depth

**"How does your RAG system work?"**

"I built a Time-Aware RAG system with a two-stage retrieval pipeline. First, I use SentenceTransformers to embed queries and documents into 384-dimensional vectors. These vectors are indexed in FAISS for fast similarity search using L2 distance. When a query comes in, I retrieve the top candidates from FAISS, then apply temporal filtering based on the query date and document validity windows stored in a JSON metadata store. Only documents valid at the query date are passed to Gemini 2.5 Flash for answer generation. This architecture prevents time-based hallucinations and ensures historically accurate answers."

**"Why did you choose FAISS over a vector database?"**

"For this project, I wanted to understand the fundamentals without abstraction layers. FAISS gives me direct control over indexing and search operations, which is valuable for learning. It's also extremely performant for small-to-medium datasets. In production, I'd migrate to Pinecone or Weaviate for distributed search and better scalability, but FAISS was perfect for demonstrating core concepts and rapid iteration during development."

### **"How do you handle temporal conflicts?"**

"The system enforces temporal validity at the metadata level. Each document has explicit valid\_from and valid\_to dates. During retrieval, I filter candidates using simple date comparisons before sending context to the LLM. This ensures the LLM never sees conflicting information from different time periods. For edge cases like overlapping validity windows, the system returns all valid documents and lets the LLM synthesize the answer, mentioning the different validity periods if needed."

## **Architecture & Design**

### **Why separate concerns into multiple modules?**

"I followed the separation of concerns principle. The embedding module handles vector generation, the vector store manages similarity search, the metadata store tracks temporal information, and the retriever orchestrates everything. This modular design makes each component testable in isolation, easy to swap (e.g., JSON → PostgreSQL), and simple to understand. It also follows production patterns where you'd have separate services for embeddings, vector search, and metadata management."

### **How would you scale this system?**

"Current architecture supports a few thousand documents. For production scale, I'd:

1. Migrate metadata from JSON to PostgreSQL with proper indexing on dates
2. Move FAISS to a vector database like Pinecone for distributed search
3. Add Redis caching for frequently accessed documents
4. Implement batch processing for document uploads
5. Use async workers for embedding generation
6. Add query result caching based on (query, date) pairs
7. Implement proper logging and monitoring"

## Problem-Solving

### What were the hardest technical challenges?

"Three main challenges: First, figuring out the right retrieval strategy. I initially retrieved only k documents, but many were filtered by time, returning too few results. Solution: over-fetch k\*3 candidates and filter. Second, preventing time-based hallucinations. Regular RAG would return all similar docs regardless of date. Solution: filter before LLM, not after. Third, managing model loading times. Embedding model takes 2-3 seconds to load. Solution: singleton pattern to load once per application lifecycle."

## COMMON ISSUES & TROUBLESHOOTING

### Issue 1: PyTorch Not Installing

**Symptom:** ERROR: Could not find a version that satisfies the requirement torch

**Cause:** Python 3.13 doesn't have PyTorch builds for all versions

**Solution:**

```
bash
Option 1: Update requirements.txt
torch>=2.6.0 # Instead of torch==2.5.1
```

```
Option 2: Downgrade Python
brew install python@3.11
python3.11 -m venv venv
```

### Issue 2: Gemini API Error

**Symptom:** 404 models/gemini-2.0-flash-exp is not found

**Cause:** Model name doesn't exist or API key issue

**Solution:**

```
python
Use stable model name
self.model = genai.GenerativeModel('gemini-2.5-flash')

Verify API key in .env
GEMINI_API_KEY=your_actual_key_here
```

### Issue 3: Empty Search Results

**Symptom:** Query returns no documents

**Cause:** Time filter too restrictive or no documents uploaded

**Solution:**

```
bash
Check if documents exist
curl http://localhost:8000/api/documents

Check if query_date is within any document's validity
If all documents valid 2022-2023, querying 2024 returns nothing
```

### Issue 4: FAISS Index Not Saving

**Symptom:** Index empty after restart

**Cause:** Not calling save() or permission issues

**Solution:**

```
python
Ensure save is called after add
retriever.add_document(...) # This calls save internally

Check permissions
ls -la data/faiss_index/
```

### Issue 5: Module Not Found Errors

**Symptom:** ModuleNotFoundError: No module named 'app'

**Cause:** Running from wrong directory

**Solution:**

```
bash
```

```
Always run from backend/ directory
```

```
cd backend/
```

```
python -m app.main
```

```
NOT: python app/main.py
```



## PROJECT CHECKLIST

### Setup Phase

- Python 3.11+ installed
- Virtual environment created and activated
- All dependencies installed from requirements.txt
- Gemini API key obtained and added to .env
- All directories created (app/, core/, llm/, data/)
- All **init.py** files created

### Implementation Phase

- app/config.py implemented
- app/models.py implemented
- core/embeddings.py implemented
- core/vector\_store.py implemented
- core/metadata\_store.py implemented
- core/retriever.py implemented
- llm/gemini\_client.py implemented
- app/main.py implemented
- app/api/routes.py implemented

## Testing Phase

- Server starts without errors
- Health check returns "healthy" at <http://localhost:8000>
- Swagger docs accessible at <http://localhost:8000/docs>
- Successfully upload 3+ documents
- Query with date 1 returns correct documents
- Query with date 2 returns different documents
- List documents endpoint works
- Stats endpoint shows correct counts

## Verification Phase

- Same query with different dates gives different answers
- LLM generates natural language responses
- Sources are included in response
- Time filtering works correctly
- System persists data across restarts

## LEARNING OUTCOMES

After completing this project, you should understand:

### Core Concepts

-  How embeddings convert text to vectors
-  How vector similarity search works (FAISS)
-  What RAG is and why it's useful
-  Why temporal information matters in real systems
-  How to prevent LLM hallucinations through retrieval

### Technical Skills

-  FastAPI web framework
-  Pydantic data validation



- ☒ SentenceTransformers for embeddings
- ☒ FAISS for vector search
- ☒ Google Gemini API integration
- ☒ Async Python programming
- ☒ REST API design

## Engineering Practices

- ☒ Modular code architecture
- ☒ Separation of concerns
- ☒ Singleton design pattern
- ☒ Error handling and logging
- ☒ Environment-based configuration
- ☒ API documentation (Swagger)

## System Design

- ☒ Two-stage retrieval (similarity + filtering)
- ☒ Stateful vs. stateless components
- ☒ Data persistence strategies
- ☒ Scalability considerations
- ☒ Production-ready patterns

## NEXT STEPS (OPTIONAL ENHANCEMENTS)

### Phase 2: Database Migration

- Replace JSON with PostgreSQL
- Add proper indexing on dates
- Implement connection pooling
- Add database migrations

### Phase 3: Advanced Features

- Batch document upload

- Date range queries ("What changed 2022-2024?")
- Document versioning
- Full-text search
- Query caching

## Phase 4: Production Readiness

- Docker containerization
- Environment-specific configs (dev/staging/prod)
- Logging and monitoring
- Rate limiting
- Authentication/authorization
- CI/CD pipeline

## Phase 5: Frontend

- React/Next.js UI
- Document upload interface
- Query interface with date picker
- Source visualization
- Analytics dashboard

## ADDITIONAL RESOURCES

### Documentation

- FastAPI: <https://fastapi.tiangolo.com/>
- FAISS: <https://github.com/facebookresearch/faiss>
- SentenceTransformers: <https://www.sbert.net/>
- Google Gemini: <https://ai.google.dev/docs>

### Learning Materials

- RAG Papers: "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"
- Vector Databases: Pinecone, Weaviate, Qdrant documentation

- Embeddings: "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks"

## CONCLUSION

You've built a production-quality Time-Aware RAG system from scratch! This project demonstrates:

- Deep understanding of RAG architecture
- Practical ML engineering skills
- Real-world problem-solving
- Clean, maintainable code
- Interview-ready technical depth

**Key Achievement:** You can now explain how modern AI systems work under the hood, not just use high-level abstractions!